

Pró-Reitoria Acadêmica
Escola de Educação, Tecnologia e Comunicação
Curso de Bacharelado em Engenharia de Software
Trabalho da Disciplina de Teste de Software

CashLens - MVP de Gestão Financeira

Plano de Testes

Versão 1.0

Autores: João Marcos Azevedo Cruz e João Pedro Tavares Teixeira

Histórico de Revisão

Data	Versão	Descrição	Autor
02/05/2026	1.0	Elaboração inicial do planejamento de testes do projeto CashLens	Grupo de Teste 9

1. INTRODUÇÃO

Este documento apresenta o planejamento de testes do projeto CashLens, sistema de gestão financeira desenvolvido para apoiar o controle de receitas, despesas, vendas, importações financeiras, indicadores gerenciais e acompanhamento de resultados.

O objetivo deste planejamento é definir, antes da execução dos testes, quais funcionalidades serão avaliadas, quais tipos de teste serão aplicados, quais recursos serão necessários, quais ferramentas serão utilizadas e quais produtos deverão ser gerados ao final da atividade.

O planejamento considera testes automatizados no backend, testes de integração com banco de dados isolado, validações de API por Postman e Swagger, além de verificações manuais no frontend por meio de navegador. Dessa forma, busca-se cobrir tanto regras internas do sistema quanto fluxos percebidos pelo usuário final.

2. ESCOPO

O escopo deste plano contempla as principais funcionalidades do CashLens relacionadas ao backend Spring, à interface React e aos fluxos de integração entre usuário, API, autenticação e banco de dados.

Serão considerados os seguintes pontos:

- autenticação e autorização com Keycloak e JWT;
- leitura do usuário autenticado e controle de acesso aos dados financeiros;
- importação de arquivos financeiros aceitos pelo sistema;
- validação de integridade em receitas, despesas, vendas, custos e quantidades;
- gravação, consulta e histórico de importações;
- operações financeiras manuais de vendas, despesas e ajustes;
- consulta de indicadores financeiros, DRE, dashboard, waterfall e visão operacional;
- validação da camada de IA controlada pelo backend, considerando segurança, roteamento semântico, tools financeiras, modelos permitidos e respostas validadas;
- validação manual da experiência de uso no frontend, navegação, responsividade e comportamento visual.

3. REFERÊNCIAS

As referências utilizadas para a elaboração deste planejamento são:

- Planejamento de testes de software - <https://medium.com/cwi-software/planejamento-detestes-de-software-d2c1a1dd0951>
- Documentação do Spring boot: <https://spring.io/>
- Teste com containers: <https://docs.spring.io/spring-boot/reference/testing/testcontainers.html>
- Keycloak: <https://www.keycloak.org/2017/05/easily-secure-your-spring-boot>

4. PLANEJAMENTO DOS TESTES

O planejamento dos testes foi organizado para combinar validações unitárias, validações de integração, validações de API e validações manuais de interface. A estratégia principal consiste em testar primeiro as regras de negócio de forma isolada, depois os fluxos que dependem de banco de dados realista e, por fim, os endpoints acessados pelo usuário autenticado. A interface web será validada por navegador, considerando que o objetivo neste momento é confirmar usabilidade, navegação e comportamento visual. Os testes automatizados do backend serão planejados em dois grupos principais:

- testes sem banco real, executados com JUnit 5 e Mockito, focados em serviços, validações, regras de negócio, segurança local e comportamento determinístico;
- testes com banco dedicado via Testcontainers, executados com PostgreSQL/pgvector em container temporário, Flyway e contexto Spring, focados em integração real com persistência.

Os testes de API serão planejados com Postman e Swagger. O Swagger será utilizado para consulta do contrato dos endpoints, parâmetros esperados e formatos de request/response. O Postman será utilizado para executar chamadas autenticadas e não autenticadas, validar códigos HTTP, enviar payloads inválidos e registrar evidências.

Os testes de frontend serão planejados como testes manuais exploratórios guiados por roteiro. Eles serão realizados em navegador atualizado, observando telas, componentes, fluxos de navegação, upload, estados visuais, mensagens de erro, responsividade e integração com o backend.

5. ITENS DE TESTE

Os itens a serem avaliados neste plano são:

Item	Descrição
Autenticação	Validar acesso autenticado, bloqueio sem token e leitura correta das informações do usuário.
Autorização	Confirmar que os dados financeiros retornados pertencem ao usuário autenticado.
Importação financeira	Validar envio de arquivos aceitos pelo sistema, pré-visualização, aplicação e tratamento de erros.
Integridade financeira	Validar bloqueio de valores incoerentes, como quantidades, preços, custos ou despesas inválidas.
Rollback	Validar reversão confiável de dados importados quando aplicável.
Histórico de importação	Verificar registro e consulta de eventos de importação.
Consultas financeiras	Validar DRE, indicadores, resumo operacional, dashboard e waterfall.

Item	Descrição
Ajustes manuais	Validar criação e regras de entradas manuais de vendas e despesas.
IA do CashLens	Validar modelo permitido, segurança, resposta sanitizada, contrato semântico, uso de tools e fallback financeiro.
API	Validar endpoints via Postman e Swagger, incluindo cenários de sucesso, erro e autenticação.
Frontend	Validar navegação, responsividade, usabilidade e comunicação visual das telas principais.

6. TIPOS DE TESTES

6.1 Testes Unitários

Os testes unitários serão aplicados principalmente na camada de serviços, validadores, parsers, formataadores e componentes de apoio. O objetivo é verificar pequenas unidades de comportamento sem depender de banco real, rede externa ou execução completa da aplicação.

Serão utilizadas as ferramentas JUnit 5 e Mockito. O JUnit será responsável pela definição e execução dos cenários. O Mockito será utilizado para simular dependências, repositórios e serviços auxiliares quando o objetivo do teste for validar apenas a regra local.

Exemplos de comportamentos planejados:

- validar regras de ajustes manuais;
- validar parser de arquivos financeiros;
- validar integridade de dados antes da persistência;
- validar normalização e segurança de respostas da IA;

6.2 Testes de Serviço

Os testes de serviço serão planejados para validar fluxos internos mais completos, eles verificam coordenação entre classes de negócio, orquestração de importação, geração de preview, histórico, rollback, previsão, simulação e leitura financeira. Esse tipo de teste ajuda a identificar falhas de regra de negócio que não aparecem em testes unitários muito pequenos.

6.3 Testes de Integração com Testcontainers

Os testes de integração serão planejados com Testcontainers, utilizando um banco PostgreSQL/pgvector dedicado e temporário para a suíte de testes. Essa abordagem evita dependência do banco local e reduz risco de interferência por dados preexistentes. O container de teste permite executar migrações Flyway, persistir registros reais e validar consultas, constraints, índices, relacionamentos e fluxos de importação em ambiente controlado. Os cenários planejados incluem:

- criação do schema via migrações;
- importação de dados financeiros;

- pré-visualização e aplicação de importações;
- conflitos de processamento;
- rollback de dados;
- persistência e leitura posterior;
- endpoints de ingestão com MockMvc e JWT simulado.

6.4 Testes de API com Postman e Swagger

Os testes de API serão planejados para validar a comunicação externa com o backend Spring.

O Swagger será utilizado para:

- consultar endpoints disponíveis;
- conferir parâmetros obrigatórios;
- validar modelos de request e response;
- apoiar a montagem dos cenários de teste.

O Postman será utilizado para:

- obter e utilizar token JWT quando necessário;
- executar requisições autenticadas;
- verificar bloqueio de requisições sem token;
- enviar payloads válidos e inválidos;
- testar endpoints financeiros, ingestão, preview e ajustes manuais;

6.5 Testes de Segurança Funcional

Os testes de segurança funcional serão planejados para cobrir riscos práticos ligados ao uso do sistema.

Serão avaliados:

- bloqueio de acesso sem token;
- interpretação correta de roles do Keycloak;
- isolamento dos dados do usuário autenticado;
- validação de arquivos enviados pelo usuário;
- prevenção de conteúdos perigosos em exportações;
- bloqueio de tentativas de acesso a dados de outro usuário pela IA;
- bloqueio de vazamento de prompts, tokens, identificadores internos e SQL.

6.6 Testes da Camada de IA

Os testes da camada de IA serão planejados considerando que o CashLens utiliza uma arquitetura em que o backend controla segurança, contexto, tools, validação e fallback.

Os testes não devem depender de chamadas reais ao provedor de IA para validar regras internas. Sempre que possível, as dependências externas serão simuladas, permitindo testar de forma determinística:

- catálogo de modelos permitidos;
- roteamento semântico;
- resolução de perguntas de continuidade;
- normalização semântica;
- validação de tools financeiras;
- executor financeiro;
- bloqueio de informações sensíveis;

6.7 Testes de Frontend em Navegador

Para o frontend, o planejamento considera testes manuais em navegador. Essa decisão foi tomada porque a etapa atual busca validar experiência do usuário, organização visual, responsividade e fluxo de telas, e não criar uma suíte automatizada end-to-end.

Serão observados:

- carregamento das telas principais;
- comportamento visual em desktop e resoluções menores;
- feedbacks de loading, erro e sucesso;
- interação com filtros;
- consistência de layout, textos e componentes.

6.8 Testes de Regressão

Os testes de regressão serão planejados para confirmar que alterações futuras não quebrem comportamentos já validados. A cada correção ou nova funcionalidade, a suíte automatizada relevante deverá ser executada novamente.

Os principais candidatos para regressão são:

- importação e preview;
- validações financeiras;
- segurança de upload;
- autenticação;
- cálculos financeiros;
- camada de IA;
- endpoints protegidos;

7. RECURSOS NECESSÁRIOS

7.1 Recursos Humanos

Papel	Envolvimento estimado	Atividades
Integrantes do grupo	1 semana	Planejar, executar, registrar evidências e analisar os testes.

7.2 Recursos Computacionais

Recurso	Detalhamento	Quantidade
Computador	Ambiente local para execução do projeto e registro das evidências	1
Java 17	Versão utilizada para compilar e executar testes do backend	1
Maven Wrapper	Ferramenta de build e execução dos testes Spring	1
Docker	Necessário para execução dos containers de teste	1
PostgreSQL/pgvector em Testcontainers	Banco dedicado e temporário para testes de integração	1
Keycloak local	Serviço de autenticação para testes manuais e Postman	1
Node.js	Ambiente de execução do frontend React/Vite	1
Navegador web	Validação da interface e dos fluxos de usuário	1

7.3 Ferramentas de Teste

As seguintes ferramentas serão usadas para o teste.

Ferramenta	Função no planejamento
JUnit 5	Definição e execução dos testes automatizados.
Mockito	Simulação de dependências nos testes unitários e de serviço.
Spring Boot Test	Apoio aos testes com contexto Spring.
Testcontainers	Subida de banco PostgreSQL/pgvector dedicado para integração.
Postman	Execução manual de endpoints e registro de evidências.
Swagger/OpenAPI	Consulta do contrato da API.
Docker	Execução dos serviços auxiliares e containers de teste.
Navegador Web	Validação manual do frontend.

7.4 Capacitação

Antes da execução, os participantes deverão alinhar:

Treinamento	Detalhamento
Escopo funcional que será testado	Alinhamento antes da execução dos testes.
Critérios de sucesso e falha	Definição do que caracteriza aprovação ou reprovação.
Comandos de execução dos testes automatizados	Padronização da execução no ambiente do grupo.
Obtenção e uso do token JWT no Postman	Preparação para chamadas autenticadas.
Classificação de defeitos	Padronização de severidade, prioridade e descrição.
Registro de evidências	Definição do padrão de prints, arquivos e observações.
Ambiente necessário	Alinhamento de backend, banco, Keycloak e frontend.

8. CRONOGRAMA DE TESTES

Atividade	Duração estimada	Responsável
Planejamento dos testes	3 horas	Integrantes do grupo
Revisão do escopo e critérios	1 hora	Integrantes do grupo
Preparação do ambiente	1 hora	Responsável técnico
Elaboração dos cenários automatizados	4 horas	Integrantes do grupo
Elaboração dos cenários Postman/Swagger	3 horas	Integrantes do grupo
Execução dos testes unitários e de serviço	4 horas	Integrantes do grupo
Execução dos testes de integração	3 horas	Integrantes do grupo
Execução dos testes de API	2 horas	Integrantes do grupo
Validação manual do frontend	2 horas	Integrantes do grupo
Registro de defeitos	1 hora	Integrantes do grupo
Consolidação do relatório final	2 horas	Integrantes do grupo

9. RISCOS E RESTRIÇÕES

Os principais riscos e restrições identificados são:

- dependência de Docker funcionando corretamente para testes com Testcontainers;
- tempo maior de execução nos testes de integração, por envolver subida de container e migrações;

- dependência de Keycloak local para testes manuais autenticados;
- configuração incorreta de variáveis de ambiente do backend ou frontend;
- dados financeiros insuficientes para alguns cenários manuais;

10. PRODUTOS GERADOS

Ao final da atividade de testes, deverão ser gerados os seguintes produtos:

- documento de plano de testes;
- documento da História de Usuário
- roteiro de testes automatizados;
- roteiro de testes de API com Postman e Swagger;
- roteiro de validação manual do frontend;
- classificação de severidade e prioridade dos defeitos;
- evidências de teste por meio de capturas de tela.
- registro de defeitos encontrados, caso ocorram.